

Learning rate Schedulers:

Learning rate scheduler automatically changes the Learning rate during training based on defined strategy.

Schedulers help:

- * escape from local minima early.
- * Speed up convergence
- * prevent overshooting near the optimum
- * better generalization enhancement.

Types of Learning Rate Schedulers:

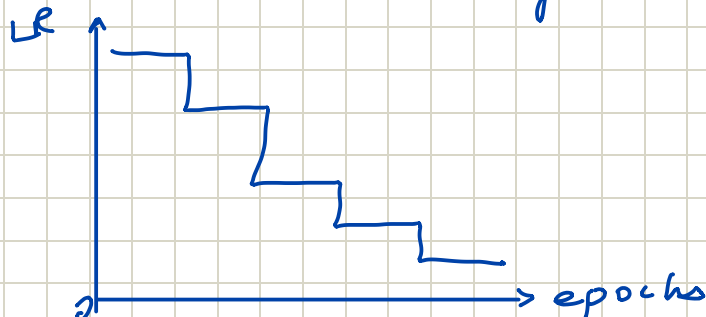
1.) Pre defined (Rule Based) Schedulers

(i) Step Decay

$$\eta_t = \eta_0 \times \text{drop}^{\lfloor \text{epoch} / \text{step} \rfloor}$$

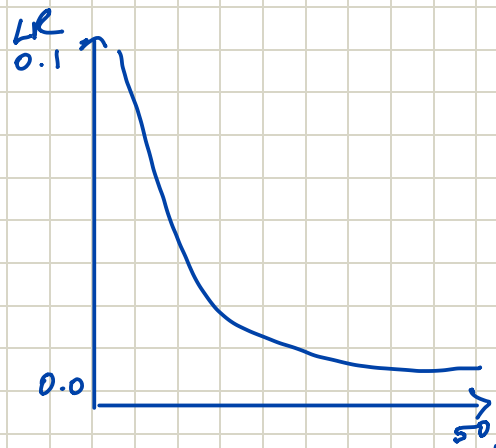
Here the Learning rate decreases by constant factor after each epochs.

Use Case: CNN like ResNet, ImageNet training.



(ii) Exponential Decay:

$$\eta_t = \eta_0 \times e^{-k \times \text{epoch}}$$



Smooth decay, ideal for fine convergence.

(iii) Polynomial Decay:

$$\eta_t = \eta_0 \times \left(1 - \frac{\text{epoch}}{\text{max-epoch}} \right)^{\text{power}}$$

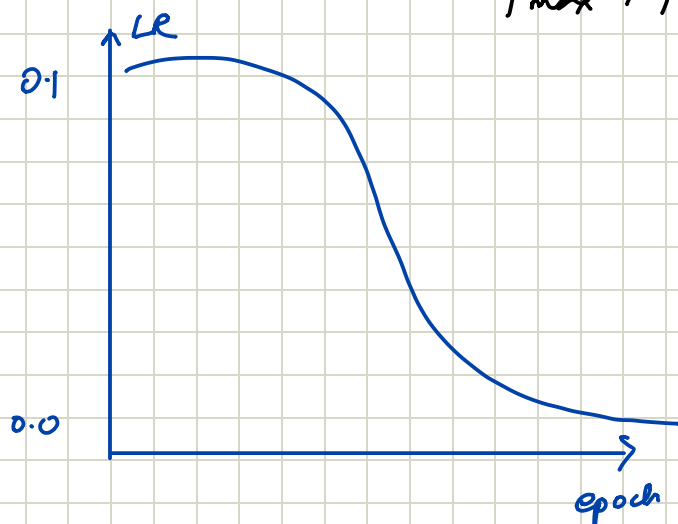
power determines how sharply it decays.

(iv) Cosine Annealing:

$$LR = LR_{\min} + 0.5(LR_{\max} - LR_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}} - T_1}{T_{\max} - T_1}\right) \right)$$

Cosine Annealing follows a cosine curve which starts high & smoothly decreasing to minimum value.

(This helps model ^{escape} from local minima & often achieves better results)



(ii) Performance based (Adaptive) schedulers.

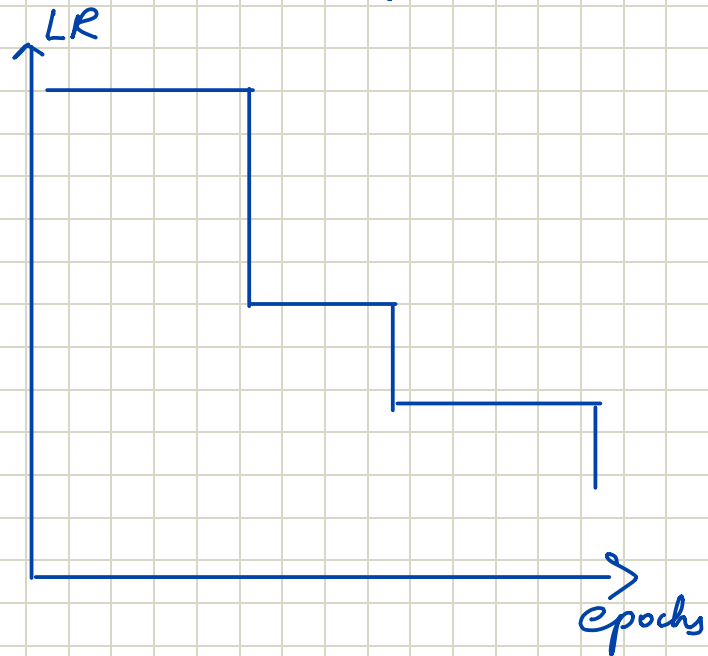
These changes Learning rate based on performance or training feedback.

(a) Reduce LR on plateau.

This takes a different approach by monitoring validation metrics and reduce the LR only if the improvement stagnates.

Pros: When you are unsure of optimal scheduling, this works excellent.

Cons: It might react slowly to make needed adjustments.



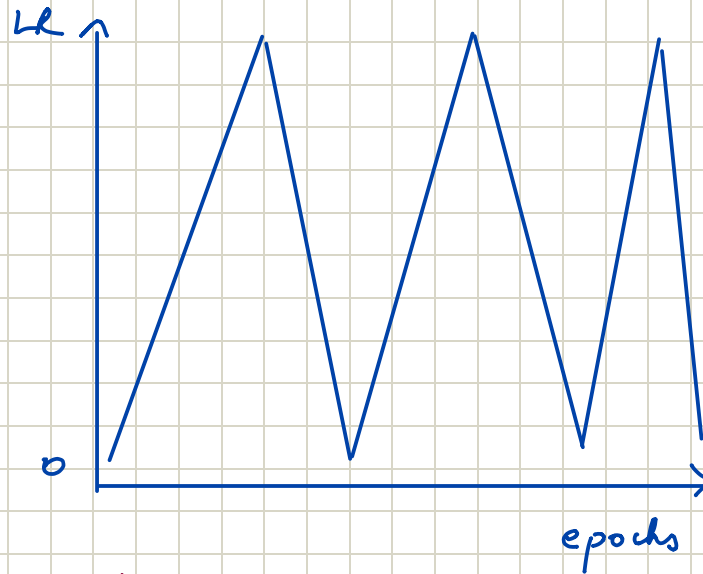
(b) Cyclical LR - Cyclic Learning Rate.

Cyclical LR oscillates between the minimum & maximum Learning rates forming triangular pattern.

This helps escape poor local minima & explore loss space effectively. They require proper tuning of minimum, maximum & cycle length parameters effectively.

When & where to use:

* When your model transition from exploration to fine-tuning, Use StepLR.



* When you want smooth, predictable decay, use ExponentialLR (for stable, continuous reduction without sudden changes that might disrupt momentum)

* When you want to escape from local minima, Use cosine annealing LR.

* When you are not sure about optimal scheduling, ReduceLROnPlateau.

* For advanced optimization techniques - CyclicalLR

Note: Always start with ReduceLROnPlateau, then experiment with others based on specific needs.

OneCycleLR: LR increases rapidly $\rightarrow$ escape from local minima

then reduces slow $\rightarrow$ fine tunes in flat minima.